5.1 Architectural model

Project: Electrical Consumption Monitor — EnergyMonitor **Record:** 3145556 — Software Analysis and Development (ADSO), SENA CIES Regional Huila **Version:** 1.0

> **Update note (10/27/2026).** This document describes the design of version 1.0, when the backend was the `energy-monitor-backend` modular monolith (Spring Modulith). That design evolved: the seven bounded contexts are now **seven microservices** behind the `en-monitor-api-gateway` API gateway (8080) — security 8082, device 8084, measurement 8085, home 8086, notification 8087, alert 8088 and recommendation 8089 —, communicating through internal REST (`/internal/**`) and RabbitMQ events with a transactional outbox, on the same shared MySQL schema (37 tables). The modular backend was retired. The hexagonal layers, contexts and patterns described here live on inside each service; the current architecture, routes, events and configuration are in the technical manual (sections 9 to 13) and the current data model in `06-base-datos`.

1. Purpose

This section defines the architecture of the system: the chosen architectural style, the organization of its layers, the separation into bounded contexts, the components that make it up, and the communication protocols between them.

The document answers four questions:

1. **How is the system organized?** — Architectural style and layers.
2. **Which components is it made of?** — Component view and technologies.
3. **How do the components communicate?** — Deployment view and protocols.
4. **What constrains the design?** — Quality attributes and verifiable architectural rules.

2. Architectural style

2.1 Adopted definition

The system adopts a **three-tier client-server** architecture, combined with:

Style	Scope	Justification
Three tiers (presentation – logic – data)	Entire system	Separates the user interface from the business logic and from persistence, allowing each layer to evolve independently
Modular monolith (`Spring Modulith`)	Backend	A single deployment, but with module boundaries verified automatically by test compilation
Hexagonal architecture (ports and adapters)	Within each module	The domain depends on nothing external: technologies are connected through interchangeable adapters
Stateless REST API	Contract between front-ends and backend	Allows web and mobile to share exactly the same contract
Publisher / Listener	Between backend contexts	Decouples the production of measurements from the generation of alerts and recommendations

> **Why a modular monolith and not microservices.** The data volume and the number of users of the project do not justify the operational cost of a distribution split into independent services. The modular monolith offers most of its benefits —single deployment, local transactions, autonomous teams within the code— without introducing network coupling or the complexity of distributed consistency. The separation into contexts is recorded in the code and verified by tests, so a future extraction is viable module by module.

2.2 Three-tier architecture

Figure 3. *General three-tier architecture of the system.*

Layer	Technology	Responsibility	Location of the code
Presentation	React 19 (web) · Expo SDK 57 (mobile)	User interface, form validation, session handling and consumption of the REST API	`Energy-Monitor-application/src` · `Energy-Monitor-Mobile/src`
Logic	Java 21 + Spring Boot 4.1.1 + Spring Modulith 2.1.1	Authentication, authorization, use cases, business rule validation and event publishing	`energy-monitor-backend/src/main/java/com/energymonitor`
Data	MySQL 8.4 + Spring Data JPA + Liquibase	Persistence of the 29 tables, versioned schema and aggregate queries	`src/main/resources/db/changelog`
Cross-cutting	Docker Compose · Mosquitto (MQTT) · Mailpit · Actuator	Database, measurement transport, test email and health verification	`docker-compose.yml` · `application.yaml`

3. Component view

3.1 Backend — seven bounded contexts

The backend is a Spring Boot application organized into **seven bounded contexts**. Each one explicitly declares its allowed dependencies in its `package-info.java` file.

Figure 4. *Map of the backend contexts and their allowed dependencies.*

Context	Responsibility	Allowed dependencies	Status
`security`	Registration, authentication, sessions, passwords, profile, RBAC, auditing	None	Active
`home`	Homes, members, favorites, access code, thresholds	`security`	Active
`device`	Measuring devices, API key, connection status, provisioning	`home`, `measurement`	Active
`measurement`	Recording and analysis of measurements, consumption levels, statistics	`device`	Active
`alert`	Automatic alert generation and inbox for consultation and resolution	`home`, `device`, `measurement`	Active
`recommendation`	Savings recommendations through consumption analysis	`device`, `measurement`	Active
`notification`	Email and Web Push delivery, delivery logging	`home`, `security`, `alert`, `recommendation`	Active

> **The boundaries are executable, not documentary.** The `ModularityTests` class invokes `ApplicationModules.verify()`, which analyzes the compiled classes and fails if any module imports an internal class of another module. The architectural rule does not depend on team discipline: if someone breaks it, the build fails.

3.2 Contract between contexts

Each context exposes a public contract in its `api/` subpackage, declared with the `@NamedInterface` annotation:

Public interface	Module	Purpose
`UserProfileQuery`, `PasswordResetDeliveryPort`, `EmailVerificationDeliveryPort`	`security::api`	Profile, recovery email and verification
`HomeApi`, `HomeThresholdsDto`	`home::api`	Query of homes and thresholds from other contexts

Public interface	Module	Purpose
`DeviceApi`, events `DeviceLinked`, `DeviceUnlinked`, `DeviceConnectivityLost`, `DeviceConnectivityRestored`	`device::api`	Query of devices and publishing of connectivity events
`MeasurementApi`, `MeasurementRecorded`, `ConsumptionLevelDto`, `HourlyEnergyDto`, `RiskConsumption`	`measurement::api`	Query of measurements and publishing of the central event
`AlertApi`, `AlertRaised`, `AlertType`, `AlertStatus`	`alert::api`	Query of alerts and publishing of generated alerts
`RecommendationCreated`, `RecommendationType`, `RecommendationStatus`	`recommendation::api`	Publishing of generated recommendations
(no public interface)	`notification`	Only publishes messages; exposes no contract

Figure 5. *Dependency flow between contexts: only through public interfaces and events.*

3.3 Internal structure of a context

All modules repeat the same organization, derived from the hexagonal architecture:

```
<module>/
├── package-info.java        @ApplicationModule(allowedDependencies = {...})
├── api/                     @NamedInterface — public contract toward other modules
├── domain/model/            Aggregates, value objects, enumerations, Preconditions
├── application/
│   ├── port/in/             Use case interfaces (INPUT port)
│   ├── port/out/            Persistence and service interfaces (OUTPUT port)
│   ├── command/             Input records of the use cases
│   ├── result/              Output records of the use cases
│   ├── usecase/             Implementations of the input ports
│   └── exception/           Application exceptions
├── adapter/
│   ├── in/web/              @RestController, dto/, exception handlers
│   ├── in/event/            Listeners (@EventListener)
│   ├── in/mqtt/             MQTT consumers
│   ├── in/scheduler/        Scheduled tasks (@Scheduled)
│   ├── out/persistence/     JPA entities, repositories, mappers, adapters
│   ├── out/security/        Hashers, generators
│   ├── out/client/          Adapters toward the API of another module
│   ├── out/event/           Event publishers
│   ├── out/generator/       Identifier generators
│   ├── out/mqtt/            Publishing of commands to the device
│   └── out/push/            Web Push
└── infrastructure/          @Configuration and Spring orchestration
```

3.4 Ports and adapters

A **port** is an interface that declares what the module needs; an **adapter** is the concrete implementation that connects that port to a technology.

Direction	Location	Port example	Adapter example
Input port	`application/port/in`	`RegisterUser`, `CreateHome`, `RegisterMeasurement`	`RegisterUserService`, `CreateHomeService`
Output port	`application/port/out`	`UserPersistencePort`, `PasswordHasherPort`	`UserPersistenceAdapter`, `BCryptPasswordHasherAdapter`

Port volume in the system:

Module	Input ports	Output ports
`security`	17	23
`home`	12	8

Module	Input ports	Output ports
`device`	10	9
`measurement`	7	6
`alert`	10	7
`recommendation`	3	5
`notification`	5	6
Total	**64**	**64**

> **What is bought with this structure.** Thanks to ports and adapters, application tests can exercise the use cases with test doubles without starting Spring or MySQL, and cryptographic, email or persistence technologies can be replaced without touching a single line of domain code.

4. Event architecture between contexts

4.1 Publishing and listening

Event	Published by	Listened to by	Effect
`MeasurementRecorded`	`measurement`	`alert`	Generates the alert corresponding to the measurement
`DeviceConnectivityLost`	`device`	`alert`	Generates a connectivity alert
`DeviceConnectivityRestored`	`device`	—	Automatically closes the device's connectivity alerts
`DeviceLinked` / `DeviceUnlinked`	`device`	—	Notifies the linking change
`AlertRaised`	`alert`	`notification`	Sends the notice through the configured channels
`RecommendationCreated`	`recommendation`	`notification`	Sends the advice through the configured channels
`AccountDeleted`	`security`	`home`	Removes the user from all of their homes
`HomeDeleted`	`home`	`device`	Unlinks the home's devices, revokes their API keys and, after the transaction commits, sends each device the MQTT command `{"cmd":"unlink"}`
`HomeOwnershipTransferred`	`home`	—	Records the owner change; no module consumes it yet

> **Recommendations without a measurement event.** The `recommendation` module does not listen to `MeasurementRecorded`: `RecommendationScheduler` triggers it every hour (`recommendation.evaluate-interval`, default `PT1H`, with an initial delay of two minutes) and it publishes `RecommendationCreated` for each recommendation generated.

4.2 Full flow of a measurement

Figure 6. *Reactive chain: device → MQTT broker → measurement → alert → notification.*





4.3 Additional asynchronous communication

Mechanism	Class	Frequency	Effect
`@Scheduled`	`DeviceInactivityScheduler`	Every 30 s	Marks as `OFFLINE` the devices that have not reported for more than 150 s
`@Scheduled`	`RecommendationScheduler`	Every hour (2 min offset)	Evaluates the recommendation rules over all homes
`@Scheduled` + `ApplicationReadyEvent`	`AbandonedDeliverySweeper`	Every 5 min and at startup	Closes email deliveries abandoned for more than 30 min
`@Async`	`AlertRaisedListener`, `RecommendationCreatedListener`	On demand	Releases the transaction thread for the notice delivery
`ThreadPoolExecutor`	`SecurityResetExecutorConfiguration`	On demand	Processes the password recovery outside the request thread

5. Deployment view

Figure 7. *Deployment in the development environment.*

Component	Technology	Port	Communication
Web application	Vite (static assets) on any server	`5173` (development)	HTTPS/HTTP → REST API
Mobile application	Expo (Android only)	—	HTTPS/HTTP → REST API
REST API	Spring Boot 4.1.1 on Java 21	`8080`	HTTP/1.1, JSON under `/api/v1`
Database	MySQL 8.4 in a container	`3306`	JDBC with `mysql-connector-j`
MQTT broker	Mosquitto	`1883` (internal) · `8883` (devices, TLS)	MQTT 3.1.1 with authentication and ACL
Mail server	SMTP (Mailpit in development)	`587` / `1025`	SMTP with STARTTLS

5.1 Runtime image

Stage	Base image	Content
Build	`eclipse-temurin:21-jdk`	Dependency download and JAR packaging
Runtime	`eclipse-temurin:21-jre`	System user `app` with UID 10001, port 8080 exposed, maximum memory of 75 %

5.2 Configuration per environment

Variable	Purpose
`MYSQL_PASSWORD` / `MYSQL_ROOT_PASSWORD`	Database credentials
`MAIL_HOST` / `MAIL_PORT` / `MAIL_USERNAME` / `MAIL_PASSWORD` / `MAIL_FROM`	Mail server
`SECURITY_RESET_PEPPER`	Pepper of the HMAC of the recovery codes
`SECURITY_CORS_ALLOWED_ORIGINS`	Origins allowed by CORS
`SECURITY_JWT_PRIVATE_KEY_PATH` / `SECURITY_JWT_PUBLIC_KEY_PATH`	Paths of the RSA key pair
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	Google sign-in
`MQTT_BROKER_URL` / `MQTT_BACKEND_USER` / `MQTT_BACKEND_PASS` / `MQTT_CLIENT_ID`	MQTT broker
`DEVICE_MQTT_PUBLIC_HOST` / `DEVICE_MQTT_PUBLIC_PORT`	Data delivered to the device when it is provisioned
`NOTIFICATION_WEB_URL` / `NOTIFICATION_VAPID_PUBLIC_KEY` / `NOTIFICATION_VAPID_PRIVATE_KEY`	Browser push notifications
`VITE_API_BASE_URL`	API origin for the web
`EXPO_PUBLIC_API_BASE_URL`	API origin for the mobile application

> **Secrets are read from the environment, never from the repository.** The `.env` file is imported automatically through `spring.config.import`. The `.env`, `application-local.yaml` and `*.pem` files are declared in `.gitignore`.

6. Front-end architecture

6.1 Common principle

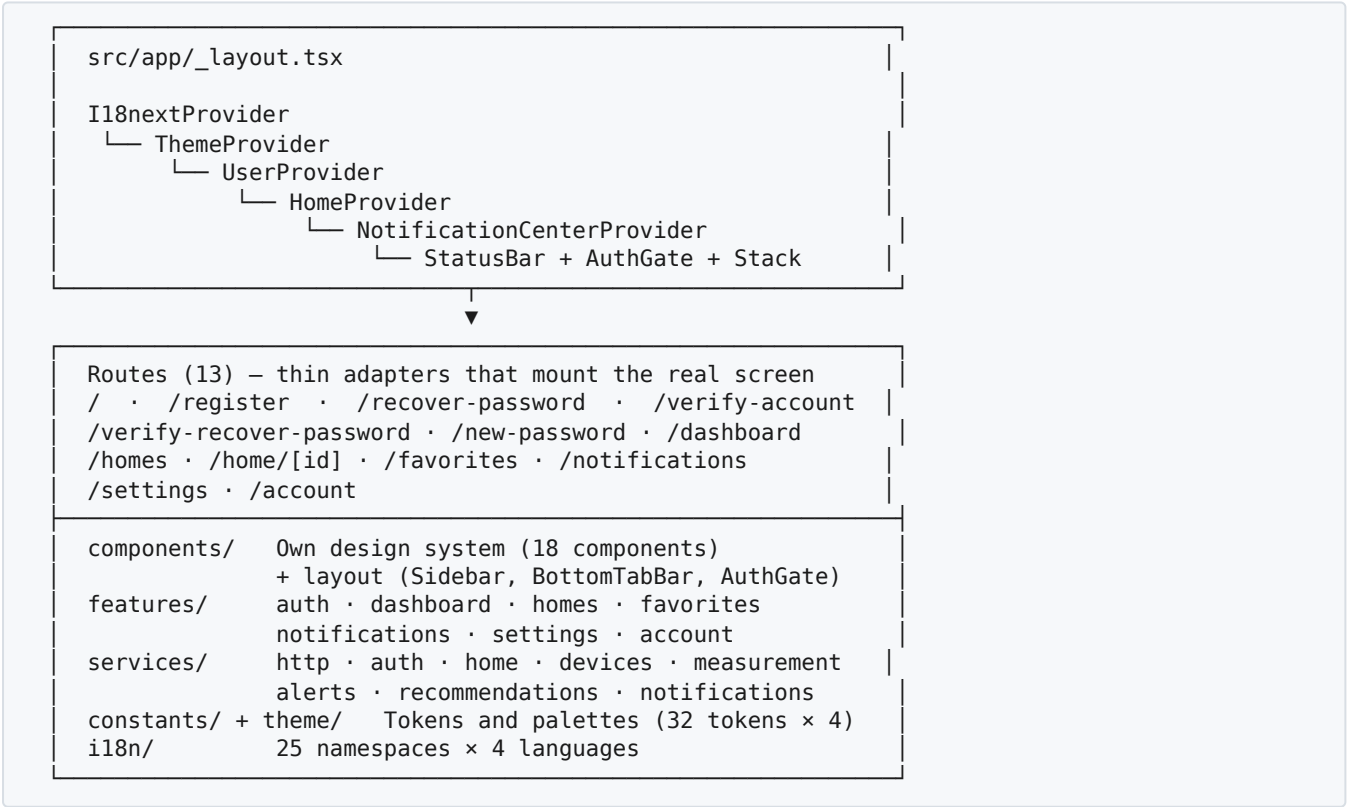
Web and mobile share the **same domain model** and the **same API contract**. Both replicate, explicitly and in documented form, the same four-theme palette, the same typographic scale and the same chart colors.

Conceptual layer	Web	Mobile
Presentation	React 19 + Vite 8 (JavaScript)	Expo SDK 57 + React Native 0.86 (strict TypeScript)
Navigation	`react-router-dom` 7, nested routes	`expo-router` 57, file-based routes
Global state	React Context (3 contexts)	React Context (4 contexts)
HTTP client	`axios` + interceptors	`axios` + interceptors
Forms	`react-hook-form` + `zod`	`react-hook-form` + `zod`
Charts	`recharts`	`react-native-chart-kit`
Styles	CSS Modules + CSS variables	`StyleSheet` + own token system
Design	Own CSS token system	Equivalent token system
Languages	4 (`es`, `en`, `fr`, `pt`)	4 (`es`, `en`, `fr`, `pt`)
Local persistence	`localStorage`	`AsyncStorage`

6.2 Web application architecture



6.3 Mobile application architecture



6.4 Mobile architecture decision

The mobile application uses a **single flat Stack** of Expo Router, without native tabs or nested stacking. The tab bar of the home detail and the side menu are **own components** built on `Animated` and local state, not router routes.

Decision	Justification
Flat stack	The product is "the equivalent web app on the phone": a side menu with the list of homes and a bottom section bar reflect that structure
Own tab bar	Prevents switching tabs from pushing a new screen onto the navigation stack, so that the "back" button exits the home and not the section
Own side menu	Allows an accordion with the list of homes that stays in sync with the global context without registering dynamic routes

7. Quality attributes and how the architecture supports them

Attribute	Requirement (requirements specification)	Architectural support
Performance	RNF-REN-01 (response < 2 s)	Aggregate database queries (`GROUP BY` per hour in UTC); on-demand queries and not on every render; lazy loading not used by scope decision
Scalability	RNF-ESC-01 (growth in users and devices)	Modular monolith with independent contexts; stateless server (JWT) allows replicating instances without a shared session
Security	RNF-SEG-01 to RNF-SEG-10	RS256 authentication with an RSA key pair; authorization through RBAC against live state; two-layer validation (Zod + Bean Validation); restrictive CORS; action auditing
Usability	RNF-USA-01 (no formal WCAG level declared)	Own design system with tokens; four themes and four languages; declarative states (`loading` / `error` / `empty` / `content`); error messages translated by contract
Availability	RNF-DIS-01 (no target set), RNF-DIS-06	Actuator `/actuator/health`; notification delivery with retry through the abandoned-delivery sweeper; degraded mode: a network outage neither closes the session nor empties the inbox
Maintainability	RNF-MAN-01 to RNF-MAN-04	Executable module boundaries; six test layers; domain code without framework dependencies; documentation in the `package-info.java`
Compatibility	RNF-COM-01 to RNF-COM-04	The backend is packaged into an image with JRE; the web is a static build; mobile is built for Android only

8. Self-imposed architectural rules

The following rules are declared in the code and are verifiable.

#	Rule	How it is verified
1	The domain does not import Spring	Review of the `domain/model` packages; module test
2	No module imports internal classes of another	`ModularityTests` (`ApplicationModules.verify()`)
3	Every database query goes through an output port	Use cases do not know Spring Data repositories
4	References between contexts are plain identifiers	Total absence of JPA association annotations
5	One `@RestControllerAdvice` per module	`SecurityExceptionHandler` is global because the filter chain also requires it
6	Use cases receive commands and return results	`*Command`, `*Query` and `*Result` types in `application/`
7	JPA entities are never exposed externally	All controllers return DTOs
8	Path identifiers are encoded on the client	`encodeURIComponent` in both front-ends
9	The error message shown to the user never comes from the server	Translation catalog per operation and status code

#	Rule	How it is verified
10	Network failures do not empty already loaded data	Notification context of both front-ends

9. Context of the analysis diagrams

This point of the documentation **does not** replace the software analysis. The relationships with point 03 are the following:

Point 03 artifact	What it contributes to this point
User stories and use cases (point 02)	Define the functionalities that the architecture must support
Web and mobile navigation diagrams	Define the screen map that is implemented in `features/`
Mockups	Define the expected visual look of each screen; they are contrasted with `05.5-graphic-interfaces.md`

10. Related artifacts

Artifact	Location
Data model	`05.2-data-model.md`
Class model	`05.3-class-model.md`
Design patterns	`05.4-design-patterns.md`
Graphic interfaces	`05.5-graphic-interfaces.md`
Architecture, class and ER diagrams	`assets/diagramas/`
Project technical manual	`10-manual-tecnico/`